MedLift-3D full run on RunPod: from renting the GPU to the end of training

This is the step-by-step guide for the full 583-patient run: what to click when renting, how to set up, the one command that runs everything, how to watch it, how to tell whether the models under- or overfit, how to save everything, how to train again, and what to do on Kaggle afterwards.

Background and the reasons behind each setting: `GPU_VASTAI.md`. Times and costs below are **estimates** scaled from our measured Kaggle run (40 patients); the real numbers appear in the logs after the first hour.

Contents

0. The plan in one table
1. Before you rent: money
2. Deploy the GPU pod: the settings on the RunPod page
3. Optional: prepare the data on a cheap CPU pod first
4. Set up the GPU pod
5. Start the full run
6. Watch it run
7. Is the model under- or overfitting?
8. After training: save everything
9. Train again
10. The rest on Kaggle
11. Troubleshooting
12. Checklist

0. The plan in one table

where	what	time (estimate)	cost (estimate)
(optional) RunPod **CPU** pod	steps 02-04: choose patients, download the CT (~64 GB), build the cases	3-5 h	< $2
RunPod **GPU** pod, 1x L40S	steps 02-07 (if not done on the CPU pod), **08 diffusion training (6 h)**, 14 detector, 09 all methods, 11 growth, 10 tables, 15 fit check, two zips	16-21 h (12-17 h after the CPU pod)	$18-24 at $1.09/h
Global volume (persistent storage)	automatic backups + the zips (+ the prepared cases with the CPU pod)	the whole run	< $1 (a few GB to ~30 GB at $0.09/GB per month, prorated)
Google Drive / laptop	the final copy of everything	0.5-2 h upload	free
Kaggle (free)	step 10 tables again, the demo (step 12)	~30 min	free

Which steps really need the GPU: 05-09, 11 and 14 run on the GPU (05 would take ~39 h on a CPU). Steps 02-04 (selecting, downloading, converting DICOM) only need CPU and network, which is

why the optional CPU pod (section 3) can do them while you are not paying for the L40S.

---

1. Before you rent: money

- **$15 is not enough.** One full run on an L40S costs about **$18-24** (16-21 h x $1.09/h + disk). If the balance runs out, RunPod **stops the pod**, and a stopped pod loses its container disk (all data and the running training). The backups on the Global volume survive, but the run is interrupted.
- **Add at least $15-20** (balance ~$30-35) for one run. Make it **about $45** if you might continue training (section 9: 3 more training hours + steps 9 / 11 again = ~$9-12 more).
- Billing is per second while the pod runs. **Terminate** the pod when finished (section 8); a stopped pod still charges for its disk.
- Optional: Settings -> Billing -> a low-balance email alert.

---

2. Deploy the GPU pod: the settings on the RunPod page

2.1 The Global volume (persistent storage)

The pod's own disk (the "container disk") is fast but **erased when the pod stops or is terminated**. The Global volume is a separate disk that survives. `runpod_run.sh` copies the models, results, outputs and logs there after every step, and writes the two final zips there.

On the Deploy page: **Persistent storage -> Add volume -> Global volume**:

field	value	why
type	**Global volume**	billed only for what is stored (~$0.09 per GB per month), not tied to one region, so any region with a free L40S works
name	`medlift`	
mount path	`/workspace`	the script writes its backups and zips to `/workspace`

Do **not** train from the volume: it is a network disk (slower, and Global volumes charge per disk operation). Training runs on the container disk; the volume only receives copies.

2.2 The pod settings

setting on the page	choose	why
Secure Cloud / Community Cloud (if shown)	**Secure Cloud**	reliable hosts; a lost host means a lost run
tab	**GPU**	
GPU	**L40S** ($1.09/h, 48 GB VRAM, 62 GB RAM, 16 vCPU)	best price per hour of training: batch 8 fits in 48 GB, bf16 support, ~2x an A40

setting on the page	choose	why
alternative GPUs	**A40** ($0.59/h, 48 GB): ~1.4-1.6x slower; use `TRAIN_HOURS=10` for the same amount of training (total ~23-32 h, ~$14-19). **A100 SXM** ($1.79/h, 80 GB): ~1.3-1.6x faster than the L40S; same command. RTX A6000 (48 GB, $0.59) works like the A40. Avoid 24-32 GB GPUs (RTX 5090, 4090): batch 8 does not fit.	
region	**Any region**	the Global volume works everywhere
GPU count	**1**	the training is sized for one GPU; two GPUs cost twice and save < 2x
template	**PyTorch 2.8.0** (`runpod/pytorch:1.4.1-cu1281-torch280-ubuntu2404` , CUDA 12.8.1)	PyTorch with CUDA is preinstalled; we only add small packages
Edit Template -> **Container Disk**	**300 GB** (default 50 GB is far too small)	the 64 GB of downloaded CT + ~45 GB of prepared data + room; with the CPU pod plan 150 GB is enough
Edit Template -> Volume / pod volume	not needed; the **Global volume** must be the one at `/workspace`	
Edit Template -> environment variables, ports	leave as they are (JupyterLab on port 8888)	
Instance pricing	**On-Demand**	Spot / interruptible pods can be taken away mid-training; savings plans need months
Start Jupyter Notebook (checkbox, if shown)	**on**	JupyterLab gives a file browser for downloads
SSH (checkbox, if shown)	optional	the Web Terminal is enough

Click **Deploy On-Demand**. When the pod shows *Running*: **Connect -> Web Terminal** (or **JupyterLab -> Terminal**). Check the disks:

```
df -h / /workspace     # "/" about 300 GB (container disk), /workspace = the Global volume
nvidia-smi             # NVIDIA L40S, 46-48 GB
```

3. Optional: prepare the data on a cheap CPU pod first

Steps 02-04 (choosing the patients, downloading ~64 GB of CT from TCIA, converting DICOM to cases) take **3-5 h** and use only CPU and network. On the L40S that is ~$3-5 of GPU time doing nothing. The two-pod plan does them on a CPU pod and leaves the result on the Global volume:

```
CPU pod:  steps 02-04  ->  /workspace/medlift_cases.tar (~23 GB, on the Global volume)  ->  terminate
GPU pod:  the normal command unpacks the tar automatically and starts at step 05
```

Honest trade-off: it saves ~$3-5 and keeps the prepared cases on the volume (a later retraining on a new pod skips the download). It costs some extra setup and a few minutes of copying. Steps 05-07 must still run on the GPU (step 05 on a CPU would take ~39 h). If the CPU tab does not let you attach the Global volume or give at least 150 GB of container disk, skip this section: the single GPU pod does everything.

3.1 Deploy the CPU pod

setting	choose	why
tab	**CPU**	
CPU type	general purpose / compute-optimized, **8-16 vCPU, >= 32 GB RAM**	step 03 downloads 8 series at once; step 04 converts one CT at a time (1-2 GB of RAM each)
container disk	**150 GB**	64 GB of DICOM + ~23 GB of cases + the tar is written to the volume
persistent storage	the **same Global volume** (`medlift`) at **`/workspace`**	the GPU pod reads the tar from there
template	PyTorch 2.8.0 if offered on the CPU tab (PyTorch runs on CPU too); otherwise a plain Ubuntu / RunPod base image	
pricing	On-Demand	

3.2 Run steps 02-04 there

```
        apt-get update && apt-get install -y tmux git python3-venv python3-pip
        tmux new -s data                        # detach: Ctrl-b d   re-attach: tmux attach -t data
        cd /root
        git clone -b claude/funny-hopper-m4j8do https://github.com/tcsquanta/medlift3d.git
        cd medlift3d
        python -c "import torch; print(torch.__version__)" || {          # no PyTorch in this image:
CPU version
          python3 -m venv /root/venv && source /root/venv/bin/activate
          pip install torch --index-url https://download.pytorch.org/whl/cpu
        }
        pip install -r requirements.txt         # torch is already there, so pip keeps it
        ONLY_CASES=1 N_PATIENTS=583 bash scripts/runpod_run.sh
```

(If you used the venv and re-attach in a new shell: `source /root/venv/bin/activate` first.)

It ends with `ONLY_CASES done: 23G packed to /workspace/medlift_cases.tar`. Check `ls -lh /workspace` and **terminate the CPU pod**. A crash in the middle: run the same command again (finished downloads and cases are skipped).

On the GPU pod (section 5) use the same `N_PATIENTS`; the script prints `unpacking the cases prepared on the CPU pod` and continues at step 05.

Volume rules (RunPod):

- A volume can only be attached **when the pod is deployed**, not later. Add the Global volume on the Deploy page of *both* pods.

- Only **one running pod** should use the volume at a time: **terminate the CPU pod before deploying the GPU pod**.
- Check on both pods that the volume really is at `/workspace` before starting: `df -h /workspace` (if a network volume is also attached, RunPod moves the Global volume to `/workspace-global`; then add `CASES_TAR=/workspace-global/medlift_cases.tar` in front of the command on both pods).

Which steps can run on the CPU pod? Only 02-04. Steps 05-07 also could, but far too slowly (step 05 alone ~39 h on a CPU vs ~20 min on the L40S), and steps 08, 14, 09 and 11 are GPU training / sampling.

---

4. Set up the GPU pod

In the Web Terminal (or JupyterLab -> Terminal):

```
        apt-get update && apt-get install -y tmux
        tmux new -s ml                          # detach: Ctrl-b d   re-attach: tmux attach -t ml
        cd /root                                # the container disk: fast; /workspace is only for
backups
        git clone -b claude/funny-hopper-m4j8do https://github.com/tcsquanta/medlift3d.git
        cd medlift3d
        pip install pydicom pylidc sqlalchemy scikit-image pyyaml gradio plotly pytest   # torch is
preinstalled
        python scripts/00_check_env.py          # must list the L40S and print the projector speed
        python -m pytest -q                     # all tests must pass (~1-3 min)
```

Why tmux: the run takes ~20 h. If the browser tab closes or the internet drops, the run keeps going inside tmux; `tmux attach -t ml` brings it back. Without tmux, closing the terminal kills the run.

---

5. Start the full run

Inside tmux, in `/root/medlift3d`:

```
        N_PATIENTS=583 TRAIN_HOURS=6 bash scripts/runpod_run.sh
```

To also keep the raw CT and the prepared data for Google Drive (section 8.3), start it like this instead:

```
        mkdir -p /root/to_drive
        KEEP_DICOM=1 WITH_DATA=1 FULL=/root/to_drive/medlift_everything.zip N_PATIENTS=583
TRAIN_HOURS=6 bash scripts/runpod_run.sh
```

5.1 What the script does, in order

order	step	what	on L40S (estimate)
1	02	choose the 583 patients with usable nodules (1,144 nodules) from the radiologist annotations	~6 min
2	03	download only those CT series from TCIA, 8 at a time (~64 GB)	1-3 h (TCIA speed)
3	04	DICOM -> cases (CT + 4-radiologist nodule masks), split patients 60 / 20 / 20 into train / cal / test; then delete the DICOM	~2.5 h

order	step	what	on L40S (estimate)
4	05	simulate the low-dose DTS (15 views, +-15 deg) for every case	15-30 min
5	06	standard DTS reconstruction (coarse SIRT) + candidate nodules	45-60 min
6	07	diffusion training pairs (DTS column -> CT column) from the **train** patients	2-3 h
7	**08**	**diffusion training, stops itself after `TRAIN_HOURS`**	**6 h**
8	14	nodule detector (train patients; best checkpoint on cal)	10-15 min
9	09	all measuring methods on 150 cal + test nodules	3-5 h
10	11	growth experiment, 20 nodules x 2 follow-ups	1.5-2.5 h
11	10	accuracy and interval tables	1 min
12	15	under/overfitting check of both models (section 7)	~5 min
13	-	`medlift_kaggle.zip` and `medlift_full_results.zip` in `/workspace`	~10 min

After **every** step it copies models, results, outputs and logs to `/workspace/medlift_backup`.

5.2 The settings and why

Settings you can put in front of the command (`NAME=value bash scripts/runpod_run.sh`):

setting	default	what it does	why this value
`N_PATIENTS`	583	how many LIDC patients	all patients that have a nodule marked by >= 3 radiologists, 4-30 mm: more patients = better generalisation; there are no more of this kind in LIDC
`TRAIN_HOURS`	6	time cap of the diffusion training	the run ends at a known time and cost; ~2x the training of our Kaggle run on a ~15x larger dataset; continuing later is always possible (section 9). A40: 10
`LIMIT_09`	150	nodules measured in step 9	enough for stable tables and >= 19 calibration nodules; all ~460 would take 10+ h
`LIMIT_11`	20	nodules in the growth experiment	40 follow-up scans: enough for H4
`JITTER` / `NEGATIVES`	3 / 2	training pairs per nodule (1 centred + shifted copies up to +-6 mm) / nodule-free lung columns per patient	variety against overfitting; the model must not expect a centred nodule nor invent one
`KEEP_DICOM`	off	`1` keeps the 64 GB of DICOM (for Google Drive)	
`WITH_DATA`	off	`1` puts the prepared data (~45 GB) into the full zip	restart at step 8 / 9 / 14 later without downloading
`BACKUP`, `BUNDLE`, `FULL`, `CASES_TAR`	in `/workspace`	where backups, zips and the CPU-pod tar go	the Global volume survives the pod
`MORE_TRAIN_HOURS`, `RETRAIN`, `RETRAIN_DETECTOR`, `DET_STEPS`, `ONLY_CASES`	off	continue / redo training, CPU pod	sections 3 and 9

Fixed inside the script (step 08 = the expensive one):

option	value	meaning and reason
`--batch-size 8`	8 training columns per step	the recipe's batch; uses roughly 35-45 GB of the 48 GB, so the GPU memory is used well without crashing. If it does not fit, it is halved automatically (8 -> 4 -> 2)
`--max-hours 6`	the time cap ("GPU cap")	the training saves, validates and stops itself after 6 h. The learning rate is constant after warm-up, so stopping at any time loses nothing and continuing later is seamless
`--steps 1000000`	an upper limit only	the time cap decides; a smaller number would block continuing later
AMP `auto` -> **bf16**	half-precision maths	~2x faster and half the memory on L40S / A100 / A40, no accuracy loss for this model
learning rate 2e-4, warm-up 500 steps, EMA 0.999	config	the smoothed (EMA) weights are used for sampling: more stable results
validation every 2,000 steps	`[val]` and `[fit]` lines	4 held-out columns are sampled (nodule-box PSNR vs the DTS input) and 32 training + 32 held-out pairs are scored at fixed noise; `best.pt` = the best nodule-box gain so far
`PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`	always	PyTorch reuses GPU memory more flexibly: fewer out-of-memory crashes
`set -euo pipefail` + `.done` markers	always	a crashed step stops the script (later steps never run on broken data); running the command again skips finished work and continues an interrupted training from `last.pt`

Why not a bigger batch on a bigger GPU? Batch 8 already keeps an L40S at ~100% use. A bigger batch processes about the same number of samples per second (fewer, larger steps) and changes the training recipe; it does not make the training finish sooner.

---

6. Watch it run

Detach with `Ctrl-b d`; come back with `tmux attach -t ml`. In a second terminal, in `/root/medlift3d`:

what	command	what you want to see
progress of any step	`tail -f logs/08_train.log` (or `05_dts`, `09_pipeline`, ...)	new lines every few seconds / minutes
training speed	first `step 50/...` lines of `logs/08_train.log`	**~0.6-0.9 s/step** on an L40S. Much slower (> 1.5 s) -> a bad host: `Ctrl-c`, terminate, new pod (billing is per second)
training quality	`grep -E "\[val\]|\[fit\]" logs/08_train.log`	every 2,000 steps; see section 7
fit report so far	`python scripts/15_check_fit.py`	works during training (reads the log)

what	command	what you want to see
GPU busy	`nvidia-smi`	utilisation 90-100%, memory ~35-45 GB during step 08
disk	`df -h / /workspace`	`/` must not reach 100% (peak ~110 GB during step 04)
every step passed	`grep -h "passed,.*failed" logs/*.log`	each line ends with `ALL PASS` (one FAIL: read that step's log before going on)
backups	`ls /workspace/medlift_backup/work/models/*`	`best.pt`, `last.pt` appear after step 08

If something stops:

- The script stopped (crash, `Ctrl-c`) but the pod still runs: run **the same command** again. Finished steps are skipped; an interrupted training continues from its last check (at most 2,000 steps lost) for `TRAIN_HOURS` more hours (set `TRAIN_HOURS` to the hours you still want).
- The pod is gone (balance, host failure): new pod with the same Global volume, section 4 setup, then

  ```
  mkdir -p work && cp -r /workspace/medlift_backup/work/. work/
  N_PATIENTS=583 TRAIN_HOURS=6 bash scripts/runpod_run.sh
  ```

  Trained models and measured nodules come back from the backup; with the CPU-pod tar steps 02-04 are skipped too; steps 05-07 run again (~3-4 h).

---

7. Is the model under- or overfitting?

Plain-English meaning:

- **Under-fitting / under-trained:** the model has not learned all it could. Its results on new patients would still improve with more training.
- **Overfitting:** the model starts memorising its training patients. It keeps getting better on them but worse on new patients.
- **Converged:** more training changes nothing; only more data or a different model would help.

The only way to tell them apart is to compare the model on **data it trained on** with **data it never saw**. Both trainings do this all the time:

model	trained on	checked on (never trained on)
diffusion (step 08)	train patients (60%), minus 10% of them	the held-out 10% of the train patients (`work/roi_pairs/val`)
detector (step 14)	train patients	the **cal** patients (20%)

The **test** patients (20%) are never used for any of these decisions; they are only for the final tables. Choosing settings by test results would make the test results meaningless.

7.1 Diffusion model (step 08): three signals in `logs/08_train.log`

```
step 24000/1000000  loss 0.0213  lr 2.00e-04  0.712s/step  time left 1.25 h
  [val] PSNR sample vs CT 21.40 dB | condition vs CT 18.95 dB  || nodule box: sample 15.10 dB |
condition 11.80 dB
  [fit] denoising loss at fixed noise (EMA model): train pairs 0.0201 | held-out pairs 0.0226  (held-
out / train 1.12)
```

signal	what it is	how to read it
`loss` (every 50 steps)	training error on random noise levels	falls almost forever and jumps around; **on its own it cannot show overfitting**
`[val] ... nodule box: sample X dB | condition Y dB`	4 held-out columns are reconstructed by the model; PSNR against the true CT in a box around the nodule, versus the plain DTS input	**gain = X - Y** must be clearly positive (Kaggle run: +3.3 dB). `best.pt` is the check with the highest gain. Noisy: changes of < 0.3 dB mean nothing
`[fit] ... train pairs A | held-out pairs B`	the model's denoising error on the same 32 training and 32 held-out pairs, with the same noise every time	**the overfitting detector.** Both falling = still learning. B rising while A falls = overfitting. B / A near 1.0-1.15 = generalises; growing above ~1.2 = memorising

The four patterns:

train pairs (A)	held-out pairs (B)	gain	verdict	what to do
falling	falling	rising	**still improving (under-trained)**	continue: `MORE_TRAIN_HOURS=3` (section 9.1)
flat	flat	flat	**converged**	done; more hours add nothing
falling	rising for >= 2 checks (> 3% above its lowest)	falling	**overfitting**	nothing urgent: `best.pt` is the peak. Better generalisation: more variety (section 9.2)
high, flat	high, flat	<= 0 after 5+ checks	**not learning**	do not train longer; check `outputs/08_diffusion/val_step*.png` and step 07

Also look at the pictures `outputs/08_diffusion/val_step*.png` (DTS input | model | true CT): the model column should get sharper and closer to the CT over the checks, with no invented nodules.

7.2 Detector (step 14): `logs/14_detector.log`

```
step 1000/6000  loss 0.1123  (3.1 min)
[val] step 1000: CPM 0.155, sensitivity at 2 FP/scan 0.21, recall top-10 0.30  <- best
```

signal	how to read it
`loss`	falls during training; cannot show overfitting by itself
`[val] ... CPM` on the cal patients (every 500 steps)	**CPM** = average detection rate at 1/8 ... 8 false alarms per scan (higher is better). `best.pt` = highest CPM (`<- best`)

CPM over the checks	verdict	what to do
peaks, then falls by > 0.03 while the loss keeps falling	**overfitting** (our 40-patient Kaggle run peaked at step 1,000)	`best.pt` is the peak; with 583 patients the peak should come later and higher. Optional: `DET_STEPS` near the peak (section 9.3)
still rising at the last check	**under-trained**	retrain with more steps: `RETRAIN_DETECTOR=d2 DET_STEPS=12000`
flat	converged	done
never above 0.05	not learning	check the data before training more

7.3 The automatic report: step 15

The script runs it at the end; you can run it any time (also during training, without `--gap`):

```
        python scripts/15_check_fit.py            # reads the logs: verdicts + curves, seconds
        python scripts/15_check_fit.py --gap      # also measures the final best.pt on train vs held-
out data (GPU, ~5 min)
        cat outputs/15_fit/fit_report.md          # verdict, reason, what to do -- for both models
```

`outputs/15_fit/fit_curves.png` shows four panels: diffusion training loss; diffusion validation (nodule-box gain + train / held-out denoising loss + where `best.pt` is); detector training loss; detector CPM on cal.

`--gap` adds a direct comparison of the **final** models:

model	healthy	memorising
diffusion	held-out / train denoising loss < 1.15 and gain difference < 1 dB	larger
detector	train CPM - cal CPM < 0.15	larger

Put `fit_curves.png` and `fit_report.md` in the report: they show the training was checked for over- and underfitting on data the model never saw.

---

8. After training: save everything

The log ends with `ALL DONE`. Nothing is lost yet, but the **container disk disappears when the pod is terminated**, so copy things off first.

8.1 What is already where

where	what	survives terminating the pod?
`/root/medlift3d/` (container disk)	everything: data (~45 GB), models, results, outputs, logs	**no**
`/workspace/medlift_backup/` (Global volume)	models (`best.pt` + `last.pt` of both), `work/results`, `outputs/`, `logs/`, `splits.json`	yes
`/workspace/medlift_kaggle.zip`	for Kaggle: both `best.pt`, splits, results, all outputs, logs, 15 test patients (~1-2 GB)	yes

where	what	survives terminating the pod?
`/workspace/medlift_full_results.zip`	**everything produced** (outputs, logs, results, both models incl. `last.pt`, configs; + data with `WITH_DATA=1`)	yes
`/workspace/medlift_cases.tar` (CPU-pod plan only)	the prepared cases (~23 GB)	yes

Do I need the Global volume at the end? It is the safety net *during* the run (automatic). It is not a good long-term archive: it costs money every month and is a beta product. The long-term home is your **Google Drive (5 TB) and laptop**. Keep the volume only while you might retrain soon (section 9; ~30 GB costs ~$2.70 per month), then delete it.

8.2 Download to the laptop (small zips)

JupyterLab -> file browser -> `/workspace` -> right-click `medlift_kaggle.zip` -> **Download**, the same for `medlift_full_results.zip`. (Files of many GB are more reliable through Google Drive.)

8.3 Copy to Google Drive with rclone (recommended)

Once per pod: connect rclone to your Drive (the Google login happens in your laptop's browser):

```
        curl https://rclone.org/install.sh | bash
        rclone config
        #   n                     new remote
        #   name> gdrive
        #   Storage> drive        (Google Drive)
        #   client_id / client_secret: press Enter (empty)
        #   scope> 1              (full access)
        #   service_account_file: Enter;  Edit advanced config? n
        #   Use auto config? n    -> it prints:  rclone authorize "drive" "..."
        #   on your laptop: install rclone (rclone.org/downloads), run that exact command, log in to
Google,
        #   copy the token it prints back into the pod
        #   Configure as Shared Drive? n;  then y (keep) and q (quit)
        rclone lsd gdrive:            # lists your Drive folders = it works
```

Results only (a few GB):

```
        rclone copy /workspace/medlift_kaggle.zip gdrive:medlift3d -P
        rclone copy /workspace/medlift_full_results.zip gdrive:medlift3d -P
        rclone check /workspace gdrive:medlift3d --one-way --include "medlift_*.zip"    # must say 0
differences
```

Everything, including the prepared data and the raw CT (if you started with `KEEP_DICOM=1 WITH_DATA=1 FULL=/root/to_drive/medlift_everything.zip`, section 5): you never need TCIA or steps 02-07 again.

```
        cd /root/medlift3d
        tar cf /root/to_drive/lidc_dicom.tar data/LIDC-IDRI && rm -rf data/LIDC-IDRI   # one big file
uploads much faster
        cp /workspace/medlift_kaggle.zip /root/to_drive/
        ls -lh /root/to_drive
        rclone copy /root/to_drive gdrive:medlift3d -P --transfers 4 --drive-chunk-size 256M
        rclone check /root/to_drive gdrive:medlift3d --one-way          # must say 0 differences
```

Started without these options? The data can still be archived (not the DICOM, already deleted): `python scripts/archive_results.py --out /root/to_drive/medlift_everything.zip --with-data`, then the two rclone lines.

Upload time: a few GB take minutes; ~160 GB take roughly 30 min to 2 h (estimate). Google allows 750 GB of uploads per day.

8.4 Finish

1. `rclone check` says **0 differences** (or the zips are on your laptop and open fine).
2. RunPod -> Pods -> **Terminate** (not just Stop).
3. RunPod -> Storage -> delete the Global volume, unless you will retrain within the next weeks.

---

9. Train again

First decide **why**, from section 7 (`outputs/15_fit/fit_report.md`), and only from the train / held-out / cal results, never from the test tables.

situation (section 7)	what to do	cost on L40S (estimate)
diffusion **still improving** at the cap	9.1 continue training	extra hours + 5-8 h for steps 9 / 11 again
diffusion **overfitting** or held-out / train gap large	9.2 train a new model with more variety	6 h + 2-3 h new pairs + 5-8 h steps 9 / 11
diffusion **converged** but results not good enough	more data or model changes; more hours do not help	-
detector under-trained / overfitting	9.3 retrain the detector only	15-30 min

9.1 Continue training (under-trained)

```
        MORE_TRAIN_HOURS=3 bash scripts/runpod_run.sh
```

Continues from `last.pt` (model, smoothed model, optimiser, step count, best score so far) for 3 more hours. `best.pt` is replaced only if a later check beats it. The old step 9 / 11 results move to `work/results/old/` (they belong to the old model) and steps 9, 11, 10, 15 and both zips run again. **Cheaper:** decide while step 08 is still running. When `[val]` / `[fit]` are still improving near the end, wait for `time budget ... reached` and `----- backup`, press `Ctrl-c`, then run the line above: steps 9 / 11 then run only once.

9.2 New model from scratch (overfitting / poor generalisation)

Overfitting is not an emergency: `best.pt` is the model from the best check, before the overfitting. To get a model that generalises **better**, give it more variety and train again:

```
        rm -rf work/roi_pairs                                  # new training pairs with more
variety
        JITTER=6 NEGATIVES=4 RETRAIN=run2 TRAIN_HOURS=6 bash scripts/runpod_run.sh
```

- `RETRAIN=run2` moves the finished model to `work/models/diffusion_before_run2` (kept, also in the zips), moves the old step 9 / 11 results to `work/results/old/`, trains a new model for `TRAIN_HOURS` and re-runs everything after it. Running the same line again after a crash continues run2 (it never moves run2 away). Use a new name for each new attempt (`run3`, ...).
- `JITTER=6`: 6 pairs per nodule (1 centred + 5 shifted) instead of 3. `NEGATIVES=4`: more nodule-free columns. More variety = harder to memorise.
- Stronger dropout (default 0.1): add the line `  dropout: 0.2` inside the existing `diffusion:` section of `configs/default.yaml` before `RETRAIN` (only for a new model, not for continuing one).
- If the old model overfit after N hours and the new one is not expected to do better, a shorter `TRAIN_HOURS` saves money; `best.pt` picks the best check either way.
- Compare the two runs with `outputs/15_fit/fit_report.md` and the `[val]` gains (held-out), then keep the better one. To go back to the old model: `mv work/models/diffusion work/models/diffusion_run2 && mv work/models/diffusion_before_run2 work/models/diffusion`, then delete `work/results/pipeline.jsonl` and `growth.jsonl` and run the script again (steps 9 / 11 are re-measured).

9.3 Detector only

```
        RETRAIN_DETECTOR=d2 DET_STEPS=12000 bash scripts/runpod_run.sh     # under-trained: more
steps
        RETRAIN_DETECTOR=d3 DET_STEPS=3000  bash scripts/runpod_run.sh     # peaked early: the
learning rate decays sooner
```

The old detector moves to `work/models/detector_before_d2`; steps 9 / 11 do not use the detector, so they are not repeated. The new test table is `outputs/14_detector/table_detection.md`.

9.4 Retraining after the pod was terminated

what you kept	on the new GPU pod (sections 2 and 4 first)	skipped
Global volume with `medlift_backup` (+ `medlift_cases.tar`)	`mkdir -p work && cp -r /workspace/medlift_backup/work/. work/`, then the command from 9.1 / 9.2 / 9.3	02-04 with the tar (05-07 run again, ~3-4 h); without the tar 02-07 run again
Google Drive `medlift_everything.zip` (with data)	`rclone config` (8.3), then `rclone copy gdrive:medlift3d/medlift_everything.zip . -P && unzip -q medlift_everything.zip && rm medlift_everything.zip`, then the command	02-07
Google Drive results zip only	as the first row, using `medlift_full_results.zip` (models, results, logs)	nothing of the data

For 9.1, `last.pt` must be there (it is in the backup and in both full zips).

---

10. The rest on Kaggle

The rented GPU did the heavy work; Kaggle (free) re-makes the tables and runs the demo.

K1. Download `medlift_kaggle.zip` (section 8). Kaggle -> Datasets -> **New Dataset** -> upload it, name it e.g. `medlift-583`.

K2. New notebook: Settings -> Accelerator **GPU T4**, Internet **on**; Add Input -> your dataset.

Cell 1 - setup

```
!git clone -b claude/funny-hopper-m4j8do https://github.com/tcsquanta/medlift3d.git
%cd medlift3d
!pip install -q pydicom pylidc sqlalchemy scikit-image pyyaml gradio plotly
```

Cell 2 - copy the bundle in (finds it wherever Kaggle put it)

```
%%bash
cd /kaggle/working/medlift3d
S=$(find /kaggle/input -name splits.json -path "*work*" | head -1)
if [ -z "$S" ]; then                        # Kaggle kept the zip packed
  Z=$(find /kaggle/input -name "*.zip" | head -1); unzip -q "$Z" -d .
else
  B=$(dirname "$(dirname "$S")"); rm -rf work outputs logs
  cp -r "$B/work" work; cp -r "$B/outputs" outputs; [ -d "$B/logs" ] && cp -r "$B/logs" logs
fi
ls work/models/diffusion work/models/detector; ls work/cases | wc -l
```

Cell 3 - evaluation tables (step 10, ~1 min)

```
!python scripts/10_evaluate.py
!cat outputs/10_evaluation/table_accuracy.md outputs/10_evaluation/table_intervals.md
!cat outputs/11_growth/table_growth.md outputs/14_detector/table_detection.md
```

Cell 4 - fit report (step 15, from the training logs; `--gap` is not possible here: the training data stayed on RunPod)

```
!python scripts/15_check_fit.py
```

Cell 5 - demo (step 12)

```
!python scripts/12_demo_app.py --selftest      # must end ALL PASS
!python scripts/12_demo_app.py --share         # open the printed gradio.live link
```

The first line should read `detector: learned detector (step 14) | calibration: yes | cases offered (test): 15`. In the demo: pick a test patient -> the detector suggests nodules -> "Reconstruct in 3D" -> tick "show radiologists' ground truth" to compare.

Cell 6 - keep the Kaggle outputs

```
!zip -qr /kaggle/working/kaggle_outputs.zip outputs
```

Then download `kaggle_outputs.zip` from the notebook's Output panel.

Send the tables (`table_accuracy.md`, `table_intervals.md`, `table_growth.md`, `table_detection.md`) and `outputs/15_fit/fit_report.md` for the report.

---

11. Troubleshooting

problem	fix
`s/step` much higher than ~0.9 on an L40S	bad host: `Ctrl-c`, terminate, deploy a new pod (backups are on the volume)
`batch per GPU reduced from 8 to 4`	fine: not enough free GPU memory, it continues with 4
`No space left on device`	`df -h /`; the container disk is too small (step 04 needs ~110 GB at its peak with the DICOM). Delete `data/LIDC-IDRI` only **after** step 04 finished; next time 300 GB
TCIA download slow / failing	run the same command again (finished series are skipped); other options: `GPU_VASTAI.md`, "If TCIA downloads are slow or blocked"
a step ends with `FAIL`	read that log in `logs/`; `docs/VERIFY.md` says what each check means. A single plotting FAIL is not fatal; a data FAIL is
pod stopped (balance)	top up, new pod with the same volume, section 6 "The pod is gone"
`rclone check` reports differences	run the same `rclone copy` again (it only uploads what is missing), then check again
Kaggle: `splits.json` not found	the dataset was not added as Input, or is still being processed: wait, then re-run Cell 2

12. Checklist

- ☐ Balance at least ~$30-35 (~$45 if you might continue training)
- ☐ (optional) CPU pod: `ONLY_CASES=1 N_PATIENTS=583 bash scripts/runpod_run.sh` -> `medlift_cases.tar` on the volume -> terminate
- ☐ GPU pod: L40S x1, On-Demand, PyTorch 2.8 template, container disk 300 GB, Global volume at `/workspace`
- ☐ Setup in tmux: clone, pip install, `00_check_env.py`, `pytest`
- ☐ `N_PATIENTS=583 TRAIN_HOURS=6 bash scripts/runpod_run.sh`
- ☐ After ~5 min of step 08: `s/step` around 0.6-0.9
- ☐ Every 2,000 steps: `[val]` gain positive and rising, `[fit]` held-out falling
- ☐ Near the 6 h cap: still improving? -> plan `MORE_TRAIN_HOURS` (section 9.1)
- ☐ `ALL DONE`; `grep -h "passed,.*failed" logs/*.log` all `ALL PASS`; `cat outputs/15_fit/fit_report.md`
- ☐ Zips to Google Drive (+ laptop); `rclone check` = 0 differences
- ☐ Terminate the pod; delete the volume when no retraining is planned
- ☐ Kaggle: dataset from `medlift_kaggle.zip`, Cells 1-6, send the tables